

中南民族大学

实验报告

实验课名称： 算法与案例分析课程设计 指导教师： 骆明

学生姓名： 周兴宇 年级专业： 大数据 23 级 学号： 202321101246

实验名称： 基于卷积神经网络的中药材图像识别与智能体查询系统

实验日期： 2026 年 6 月 10 日 实验成绩： _____

实 验 目 的	1) 掌握卷积神经网络的基本原理与迁移学习方法，理解 ResNet、MobileNet、EfficientNet 等主流图像分类模型的网络结构与设计思想； 2) 使用 PyTorch 框架完成五类中药材图像数据集的预处理、数据增强与模型训练，对比分析不同模型在小样本分类任务上的性能差异； 3) 基于 Coze 平台搭建中药材智能问答系统，构建包含权威中医药文献的知识库，实现药材功效查询、药材对比、体质推荐等智能交互功能； 4) 将图像识别模型与智能体系统集成，实现“图像识别-药材分类-智能问答”的完整流程，探索卷积神经网络在中医药文化传承中的实际应用价值。
实 验 内 容	一、系统简介 本实验旨在设计并实现一个基于卷积神经网络的中药材图像识别与智能查询系统。系统以党参、枸杞、百合、金银花、槐花五味常见中药材为研究对象，通过卷积神经网络模型对输入图片进行自动分类识别，并调用 Coze 智能体平台搭建的“本草小药师”智能助手，实现识别结果与药材专业介绍的联动查询。 二、相关理论与模型基础 1. 卷积神经网络 卷积神经网络是一种专门处理具有类似网络结构的数据而设计的深度神经网络。其核心思想是通过卷积操作实现自动提取局部特征，实现空间不变性和参数高效性。其主要结构有：输入层、卷积层、激活层、池化层、全连接层以及输出层。 (1) 输入层：药材识别中卷积神经网络的输入层输入的是经过预处理后的药材图片。图片质量的好坏直接影响着卷积神经网络整个模型的准确率，因此在输入数据时要保证输入数据的均衡性。在输入层，每个药材图片都需要转换成固定维度的矩阵。因为图片都是 RGB 彩色图片，因此图片的通道数为 3。因为本文使用的数据量较小，为保证数据的多样性和降低模型过拟合风险，本文采用旋转、缩放、裁剪、翻转以及颜色抖动等方式分别对图片进行处理。 (2) 卷积层：卷积层是卷积神经网络中最核心的部分，卷积神经网络中包含多个卷积层，同层节点之间不连接，相邻层之间才有信息传递关系，而每一个节点的输入只来自上一层数据的局部感受野。这就使得不同节点分别表示图像的不同局部，最终组合起来表示整张图片，而不需要某一个节点一次包含整张图片。卷积层还应包括激活函数。若是全部通过卷积计算对所有的卷积层进行连接，这就会导致整个网络还是线性关系。通过激活函数激活，可以将线性的结构转换成非线性的结构。卷积层前向传播计算公式为：

$$X_i^l = f\left(\sum_{i \in M} X_i^{l-1} * k_{ij}^l + b_j^l\right),$$

其中 X_i^l 表示第 l 个卷积层的第 j 个输出特征图； X_i^{l-1} 表示第 $l-1$ 层的第 i 个输入特征图，也就是当前层的输入数据； k_{ij}^l 表示第 l 层中，连接上一层输入特征图与当前层第 j 个输出特征图的卷积核。每个输出特征图可能由多个特征图通过不同卷积核组合得到； b_j^l 表示第 l 层第 j 个输出特征图对应的偏置项。 f 表示激活函数，常见的激活函数有 sigmoid、relu、tanh 等。以 Relu 激活函数为例，其计算公式为：

$$f(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

从公式可以观察发现，该函数的意义 就是为了获取当前状态的最大值。当其大于零时，Relu 函数就变成了线性函数，输入值与输出值相等；当输入小于零时，其值就均变成零。该函数具有良好的数学性质，其保证了在范数范围内模型梯度为常数，有效的解决了 sigmoid 函数存在的梯度弥散问题，并在一定程度上降低了计算成本。

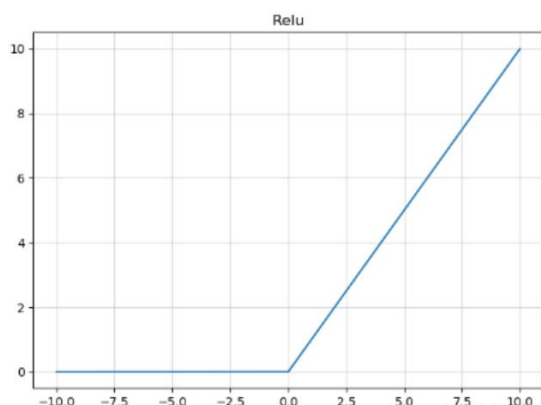


图 1 Relu 激活函数

(3) 池化层：池化层通过在输入特征图上滑动固定窗口，对窗口内的元素进行聚合操作，生成更小的特征图，从而实现特征降维和计算优化。池化层通常紧随卷积层之后，用于进一步提取抽象特征并减少模型复杂度，防止模型过拟合。常见池化层方式包括最大池化和平均池化。最大池化就是选择池化窗口内最大值作为输出，保留最显著的特征。平均池化则是计算窗口内元素的平均值输出，平滑特征并减少噪声，保留更多的背景信息。

(4) 全连接层：全连接层主要用于整合高层语义特征，输出分类或回归结果。

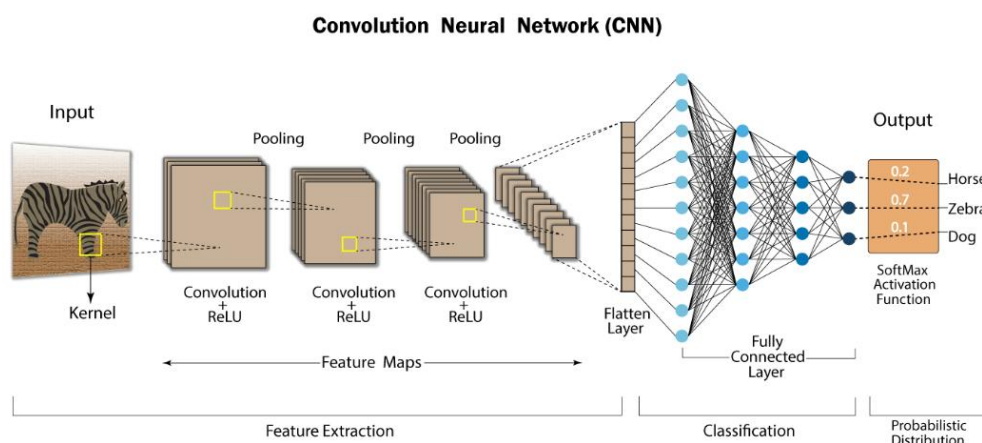


图 2 卷积神经网络示例

2. 迁移学习

迁移学习是一种机器学习方法，它利用在一个任务中学习到的知识，辅助解决另一个相关但不

同的任务。其核心目标是将单一领域或任务习得的知识、特征模式迁移至其他相关领域与问题中，提升新任务的学习效果。

从实现方式划分，迁移学习主要分为三类：基于实例的迁移、基于特征的迁移以及基于共享参数的迁移。在迁移学习中，原有知识所在的领域称为源域，待学习的新任务领域称为目标域。与传统机器学习不同，迁移学习不要求源域与目标域数据严格服从同一分布，仅需二者存在一定关联。模型可将源域数据习得的特征作为目标域的初始特征，在目标域训练过程中持续优化、筛选特征，并将有效特征应用于分类任务，提升模型整体性能。同时，该方式还能缓解模型训练初期收敛不稳定、初始准确率偏低的问题。

深度学习模型的训练通常依赖大量标注图像，充足的标注数据能够帮助模型学习到鲁棒的特征，提升复杂场景下的识别精度。但高质量标注数据获取难度大，且完整训练流程耗时久、计算成本高。针对小样本场景，迁移学习能够有效降低模型过拟合风险，减少训练开销。

常用的迁移学习方法有：

- (1) 迁移预训练模型：卷积神经网络的低层卷积层侧重提取图像底层视觉特征，高层卷积层负责挖掘抽象语义特征。可先在通用大数据集上完成模型预训练，让模型习得通用基础特征，再将预训练模型迁移至新任务继续训练。
- (2) 将预训练模型当成特征提取器：卷积神经网络具有分层特征学习的特性，由卷积层提取特征、全连接层完成分类。可直接使用预训练模型完成特征提取，将输出特征作为新任务的输入，进一步学习任务专属的深层特征。
- (3) 微调现有的预训练模型：结合新任务与源任务的相似度，对预训练模型进行适配调整。通常冻结部分底层网络参数，替换原有分类层，并对高层参数进行更新，使其适配新任务。

3. ResNet

ResNet 即残差网络，是 2015 年由微软研究院提出的深度卷积神经网络模型。传统卷积神经网络存在的网络退化问题：随着网络堆叠层数持续增加，模型训练会出现梯度消失、梯度弥散现象，模型训练准确率与测试准确率不升反降，且该问题区别于过拟合，无法通过正则化手段解决。ResNet 针对这一问题，引入残差块，使用跳跃连接让梯度可以直接传递。

$$y = F(x) + x.$$

其中， x 表示输入， $F(x)$ 是卷积支路学习得到的残差特征。ResNet 中短路直连之路不经过卷积运算，直接将输入特征传递至输出端，实现恒等映射。短路连接能够打通梯度方向传播通道，使梯度可以跳过多层卷积直接回传至浅层网络，有效缓解深层网络梯度消失问题。常用的 ResNet 包括 ResNet18、ResNet34、ResNet50 以及 ResNet101 四类结构。其中 ResNet18 和 ResNet34 采用基础残差块，参数量较小，训练速度快；ResNet50 和 ResNet101 采用瓶颈残差块，能够压缩通道维度、降低计算量，适配高精度图像处理任务。ResNet 模型具有泛化性极强的优点，是图像分类、目标检测、图像分割以及迁移学习任务中最为主流的骨干特征提取网络。

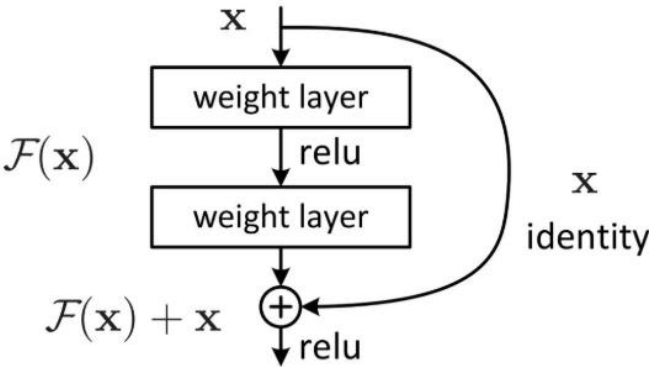


图 3 ResNet 网络残差块结构图

4. MobileNet

传统卷积神经网络具有运算量大、内存要求高以及无法在微小设备上运行的特点，而 MobileNet 能有效解决这一问题。MobileNet 是一种轻量级模型，其模块设计基于深度可分离卷积方法，目的是保持高精度的同时减少计算量和参数量。其还通过扩张系数来灵活控制通道数量，实现模型大小与精度的精细化平衡。MobileNetv1 整体结构采用类似 VGG 的直筒式设计，参数量仅为同规模传统模型的 1/32，但该版本仍存在明显缺陷：深度可分离卷积未改变通道维数，输入通道数与输出通道数相等，若上一层通道数较少，模型只能在低维空间提取低等级特征，特征复用性不足，这就很容易导致模型性能出现瓶颈。MobileNetv2 在 MobileNetv1 基础上进行了结构优化，通过倒残差与线性瓶颈设计实现了更高的准确率与计算效率。其残差模块采用“两头大、中间细”的漏斗状结构，与传统残差“先降维再升维”的顺序相反，倒残差先通过 1×1 卷积升维，让网络在高维空间完成特征学习，再通过降维操作提取特征，有效避免了低维空间下特征信息被破坏的问题。同时，模型在模块的最后一层卷积层使用线性激活函数，避免了激活函数在低维空间对特征的破坏。

5. EfficientNet

EfficientNet 是一组兼顾精度与运算效率的卷积神经网络，该模型摒弃传统网络单一调整网络深度、宽度或输入图像分辨率的优化思路，提出复合缩放策略，通过统一且合理的缩放系数，同步对网络深度、通道宽度与输入分辨率三个维度进行均衡缩放，在控制计算开销的同时最大化模型特征表达能力。EfficientNet-B0 以移动翻转瓶颈卷积模块为基础单元，延续了轻量化网络结构优势，结合残差连接机制缓解梯度消失问题。相较于 ResNet、MobileNet 等经典模型，该网络结构设计更为紧凑，参数量与计算量显著降低，同时能够实现更高的识别精度。作为系列基准模型，EfficientNet-B0 参数量小、推理速度快，综合性能优异，常被用作图像分类、目标检测等视觉任务的骨干网络，也广泛应用于迁移学习与轻量化部署场景。

6. 优化器与学习率调整

(1) Adam 优化器

Adam 优化器是一种结合动量法和 RMSProp 优点的自适应学习率优化算法，其主要应用于高效训练神经网络。Adam 优化器通过计算梯度的一阶矩和二阶矩的指数移动平均，并进行偏差校正，实现对每个参数的自适应学习率调整，从而加快模型收敛并提高训练稳定性。

- ◆ 一阶矩估计：类似 Momentum 算法，纪录梯度的指数加权平均，保留过去梯度的惯性信息，使参数更新更平滑。

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

- ◆ 二阶矩估计：类似 RMSProp，纪录梯度平方的指数加权平均，用于调整学习率，避免梯度过大或过小导致的训练不稳定。

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

- ◆ 偏差校正：由于初始时一阶和二阶矩均为 0，Adam 对其进行偏差修正，确保训练初期梯度估计不偏向 0。

$$\hat{m}_t = \frac{\hat{m}_t}{1 - \beta_1^t},$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

其中， β_1 和 β_2 分别控制一阶和二阶矩的衰减率。Adam 具有自适应学习率、训练稳定、使用范围广的特点。

(2) 权重衰减

权重衰减是一种常用的正则化方法，主要用于控制模型的复杂度，避免过拟合。其本质是在模型的损失函数中添加一个额外的惩罚项，以惩罚那些过大的权重参数。

$$L_{total}(w, b) = L_{orig}(w, b) + \frac{\lambda}{2} \|w\|_2^2,$$

其中， L_{total} 是新的总损失函数， L_{orig} 是原始损失函数， λ 是正则化系数，用于控制惩罚项的强度， $\|w\|_2^2$ 是权重的二范数，表示权重的平方和。权重衰减的作用包括防止模型过拟合以及保持模型的稳定性。通过限制权重的大小，权重衰减可以减少模型对训练数据集噪声的敏感程度，从而提高模型的泛化能力。

(3) 学习率调整

学习率调整策略通过动态控制参数更新步长来提高模型训练效率和收敛稳定性。常见的学习率调整策略有固定学习率、等间隔衰减、多步衰减、指数衰减、余弦退火、自适应调整以及 Warmup 预热。其中，余弦退火策略通过模拟退火过程中的冷却曲线来逐渐减小学习率，从而有助于模型在接近最优解时更加精细地调整权重，避免过早收敛到局部最小值。余弦退火策略的工作原理是在每次迭代中按照余弦函数的形式调整学习率。在训练开始时，学习率从一个较大值开始逐渐减小，随着迭代次数的增加，学习率的下降速度会先减慢后加快，最后再次减慢，直到达到一个预设的最小值。这种周期性变化过程可以帮助模型跳出局部最优解，增加找到全最优解的可能性。

(4) 早停

早停机制的核心思想是在模型训练过程中持续监控验证集的表现，当验证集性能不再提升或开始下降时，模型就会提前终止训练，以此来避免模型出现过拟合现象。

7. 数据增强

数据增强通常是依赖从现有数据生成新的数据点来人为增加数据量的过程。这包括对数据进行不同方向的扰动处理或使用深度学习模型在原始数据的潜在空间中生成新数据点以人为的扩充新的数据集。在实际的应用场景中，数据集采集、清洗以及标注在大多数情况下都是一个非常昂贵且费时费力的事情。利用数据增强技术就能很好的解决这一问题并且有效提升模型性能。常用的图像数据增强方法有：翻转、旋转、裁剪、缩放、平移、加噪声、模糊、色彩抖动等。

三、数据预处理

1. 数据集划分

本文主要目的是对百合、党参、槐花、枸杞以及金银花五种中药图片进行识别。数据集中共包含 892 张图片，其中百合 170 张、党参 190 张、枸杞 185 张、槐花 167 张、金银花 180 张。本文按照 80%、10%、10%的比例将数据集划分成训练集、验证集以及测试集，并将划分后的数据按照类别存入不同的文件夹下。

```
01 # 原始数据路径
02 data_dir = r'F:\药材识别\data'
03 # 目标路径（用于存放分割后的数据）
04 output_dir = r'F:\药材识别\split_data'
05 train_dir = os.path.join(output_dir, 'train')#划分后训练集数据
06 val_dir = os.path.join(output_dir, 'val')#划分后验证集数据
07 test_dir = os.path.join(output_dir, 'test')#划分后测试集数据
08 # 创建目标文件夹
09 for folder in [train_dir, val_dir, test_dir]:
10     os.makedirs(folder, exist_ok=True)
11 # 获取所有类别
12 categories = [cat for cat in os.listdir(data_dir) if os.path.isdir(os.path.join(data_dir, cat))]
13 # 遍历每个类别
```

```

14 for category in categories:
15     # 获取该类别的所有图片路径
16     cat_path = os.path.join(data_dir, category)
17     images = [img for img in os.listdir(cat_path) if img.lower().endswith(('.png', '.jpg', '.jpeg', '.gif'))]
18     # 第一次分割: 80% 训练集, 20% 剩余 (用于测试和验证)
19     train_imgs, temp_imgs = train_test_split(images, test_size=0.2, random_state=42)
20     # 第二次分割: 将剩余的 20% 分成 1:1 (测试集和验证集各占原始数据的 10%)
21     val_imgs, test_imgs = train_test_split(temp_imgs, test_size=0.5, random_state=42)
22     # 创建类别子文件夹
23     for folder in [train_dir, val_dir, test_dir]:
24         os.makedirs(os.path.join(folder, category), exist_ok=True)
25     # 复制图片到对应文件夹
26     for img in train_imgs:
27         shutil.copy(os.path.join(cat_path, img), os.path.join(train_dir, category, img))
28     for img in val_imgs:
29         shutil.copy(os.path.join(cat_path, img), os.path.join(val_dir, category, img))
30     for img in test_imgs:
31         shutil.copy(os.path.join(cat_path, img), os.path.join(test_dir, category, img))
32     print(f'类别 {category}: 训练集 {len(train_imgs)} 张, 验证集 {len(val_imgs)} 张, 测试集 {len(test_imgs)} 张')

```

2. 数据增强

本文基于数据划分得到的训练集、验证集和测试集，通过随机旋转、裁剪、翻转、颜色抖动等数据增强技术对训练集数据进行数据增强。具体流程为：

- (1) 尺度调整与裁剪：本文对图像进行统一缩放至 256×256 像素，然后将缩放后的图像随机裁剪至 224×224 像素，裁剪区域的面积占原图的比例为 0.8~1.0，宽高比控制在 0.9~1.1，这就使得模型能够学习到不同尺度与位置的目标特征。
- (2) 几何变换：通过随机旋转、随机水平翻转与随机垂直翻转，模拟不同拍摄角度与形态样本，丰富训练数据的多样性。
- (3) 色彩空间增强：引入色彩抖动操作，将图像亮度、对比度、饱和度与色调进行随机调整，模拟不同光照条件下的样本分布，提升模型对光照变化的适应能力。
- (4) 标准化处理：将图像转换为张量格式，并采用 ImageNet 预训练模型的通用均值与标准差进行归一化操作，其中均值设置为[0.485, 0.456, 0.406]，标准差设置为[0.229, 0.224, 0.225]，以统一数据分布，加快模型收敛速度。其中，使用统一的均值和标准差的核心目的是保证数据分布与预训练模型的输入分布保持一致。因为本文采用迁移学习策略，所使用的网络均是基于 ImageNet 完成预训练的，所以使用统一的均值和标准差以提高模型的收敛速度和泛化能力。

本文对验证集和测试集不采用任何数据增强操作，仅对数据将图像缩放至 256×256 像素，再通过中心裁剪获取 224×224 像素的中心区域，以消除图像边缘的无关信息的干扰。然后将图片转换成张量格式，并采用与训练集相同的均值与标准差进行归一化，保证数据分布与训练阶段一致，确保模型评估结果的可靠性。

```

01 data_transforms=[
02     # 训练集: 224x224, 增强数据增强
03     'train':
04     transforms.Compose([
05         # 调整图片大小 (注意: 数值设置太大 CPU 易跑不起来)

```

```

06     transforms.Resize([256, 256]),
07     # 数据增强（随机旋转、裁剪、水平翻转、垂直翻转、颜色抖动、灰度化）
08     # 随机旋转 30 度到-30 度
09     transforms.RandomRotation(30),
10     # 随机裁剪并缩放到 224*224，裁剪面积占 80%*100%，宽高比为 0.9: 1.1
11     transforms.RandomResizedCrop(224, scale=(0.8, 1.0), ratio=(0.9, 1.1)),
12     # 随机水平翻转（50%概率执行）
13     transforms.RandomHorizontalFlip(p=0.5),
14     # 随机垂直翻转（30%概率执行）
15     transforms.RandomVerticalFlip(p=0.3),
16     # 随机颜色抖动：调整亮度、对比度、饱和度、色调，模拟不同光照
17     transforms.ColorJitter(brightness=0.3, contrast=0.2, saturation=0.2, hue=0.1),
18     # 转换为张量
19     transforms.ToTensor(),
20     # 标准化：像素值转换为 0-1 之间
21     # 均值：[0.485, 0.456, 0.406]
22     # 标准差：[0.229, 0.224, 0.225]
23     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
24 ],
25 # 验证集：224x224，不需要做数据增强，只需要调整大小和标准化
26 'val':
27     transforms.Compose([
28         transforms.Resize([256, 256]),
29         transforms.CenterCrop(224),
30         transforms.ToTensor(),
31         transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
32     ]),
33 # 测试集：224x224（与验证集相同，不需要数据增强）
34 'test':
35     transforms.Compose([
36         transforms.Resize([256, 256]),
37         transforms.CenterCrop(224),
38         transforms.ToTensor(),
39         transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
40     ]),
41 }

```

3. 数据集加载

完成图像预处理后，本文基于 PyTorch 内置 ImageFolder 工具读取划分完成的训练集、验证集、测试集，按照预设的图像变化规则分别加载三类子集。考虑到实验设备仅具备 CPU，且硬件算力和内存读写能力有限，为避免内存溢出、CPU 算力过载导致训练中断，本文将模型批次大小设置为 16。在数据加载阶段，调用 DataLoader 构建批量数据迭代器，对训练集、验证集和测试集统一开启样本随机打乱操作。样本打乱能够有效消除数据集内部样本顺序的相关性，防止模型依据样本分布顺序学习无效特征，避免模型出现过拟合现象，保障梯度下降方向贴合全局最优解。同时，统计各子集

样本总量，并提取训练集文件夹标签作为数据集分布标签，用于后续模型分类头输出、实验指标统计。

```
01 # 批次大小：调整为 16 以适配 CPU
02 batch_size=16
03 # 创建数据集字典，应用数据增强
04 image_datasets={x:ImageFolder(os.path.join(output_dir,x),data_transforms[x]) for x in ['train','val','test']}
05 # 创建数据加载器字典（shuffle=True-每个批次打乱数据顺序）
06 data_loaders={x:torch.utils.data.DataLoader(image_datasets[x],batch_size=batch_size,shuffle=True) for x in
07 ['train','val','test']}
08 # 计算数据集大小（训练集和验证集）
09 dataset_sizes={x:len(image_datasets[x]) for x in ['train','val']}
10 # 获取类名列表（训练集的类名）
11 class_names=image_datasets['train'].classes
12 print(f'训练集大小: {dataset_sizes["train"]}, 验证集大小: {dataset_sizes["val"]}')
13 print(f'类别: {class_names}')
```

四、具体实现

1. 单模型训练

(1) 模型初始化

为缓解小样本数据集训练模型易出现过拟合问题，本文使用 ResNet18 轻量化残差网络作为特征提取模型。在模型初始化阶段选择采用 PyTorch 内置网络权重。首先，本文自定义参数梯度控制函数。该函数实现特征提取和模型微调两种迁移学习模式切换。当 feature_extract=TRUE 时将遍历网络全部卷积层、批归一化层参数，关闭参数梯度求解并冻结主干网络，实现仅训练分类层；当 feature_extract=FALSE 时，设置对网络参数梯度求解，实现对全局参数进行微调。

```
1 #冻结参数函数：控制网络参数是否参与梯度更新，实现特征提取/模型微调两种迁移学习模式切换
2 def set_parameter_requires_grad(model,feature_extract):
3     # 是否提取特征（冻结主干）
4     if feature_extract:
5         # 遍历模型所有可学习参数
6         for param in model.parameters():
7             # 标记该参数不需要梯度，反向传播时不更新
8             param.requires_grad=False
```

initial_model 函数实现对模型的初始化：首先：加载 ResNet18 模型结构以及预训练权重；接着，执行参数冻结函数完成梯度权限配置；然后，修改原模型全连接层使得输出维度变为 5，以适配五分类任务要求；最后，统一网络输入尺寸为 224×224，与图像数据处理保持一致，避免出现尺寸不匹配造成张量运算报错。完成模型初始化后，通过调用 named_parameters 接口遍历查看网络所有参数。

```
01 # 模型初始化函数
02 def initialize_model(model_name, feature_extract, num_classes, use_pretrained=True):
03     """初始化模型"""
04     # 加载 resnet18 结构以及预训练权重
05     model_fit=models.resnet18(pretrained=use_pretrained)
06     # 按策略冻结或解冻参数
07     set_parameter_requires_grad(model_fit, feature_extract)
```



```

08 #读取原全连接层输入维度
09 num_fts=model_fit.fc.in_features
10 # 将 1000 类输出层微调为 5 类新线性层
11 model_fit.fc=nn.Linear(num_fts,num_classes)
12 # 图形输入参数
13 input_size=224
14 return model_fit, input_size
15 model_fit, input_size=initialize_model(model_name, feature_extract, 5, use_pretrained=True)
16 # 使用 GPU 还是 CPU
17 model_fit=model_fit.to(device)
18 # 保存模型
19 filename='resnet18_optimized.pth'
20 # 是否训练所有层
21 params_to_update=model_fit.parameters()
22 print('Params to learn:')
23 if feature_extract:
24     params_to_update=[]
25     for name, param in model_fit.named_parameters():
26         if param.requires_grad:
27             params_to_update.append(param)
28             print('\t', name)
29     print('Only the last layer is updated.')
30 else:
31     for name, param in model_fit.named_parameters():
32         if param.requires_grad:
33             print('\t', name)
34     print('All layers are updated.')

```

(2) 训练超参数与优化策略

本文采用交叉熵损失函数作为模型损失评估准则，选择 Adam 优化器来自适应调整模型学习率。Adam 优化器能够结合一阶矩和二阶矩自适应调整每一组网络参数的学习率，具有加快模型收敛的优点。结合 CPU 算力有限、模型参数量小等实际条件限制，本文设置初始学习率为 0.0001。设置权重衰减系数为 0.0001 来约束模型参数权重值大小，以降低模型出现过拟合现象风险。本文还引入余弦退火机制动态调整学习率，防止模型陷入局部最优解。为进一步防止模型出现过拟合现象，本文使用早停策略。

```

1 # 优化器 Adam, 学习率设置为 0.0001, 权重衰减 0.0001 防止模型过拟合
2 optimizer_ft=torch.optim.Adam(params_to_update, lr=1e-4, weight_decay=1e-4)
3 # 余弦退火: 30 个 epoch 内学习率从初始值平滑下降至 0.000006
4 scheduler=torch.optim.lr_scheduler.CosineAnnealingLR(optimizer_ft, T_max=30, eta_min=1e-6)
5 # 多分类交叉熵损失
6 criterion=nn.CrossEntropyLoss()

```

(3) 模型训练

本实验将每一轮迭代分为训练与验证两阶段。每轮迭代结束后，统计训练集和验证集的平均损

失和分类准确率，并与最佳验证准确率比较。训练全部结束后，加载最优模型权重作为最终模型。

```
001 def
002 train_model(model, data_loaders, criterion, optimizer_ft, scheduler, num_epochs=30, patience=10, filename='resnet18_opti
003 mized.pth'):
004     # 纪录训练开始时间
005     since=time.time()
006     # 初始化最佳准确率
007     best_acc=0.0
008     # 确保模型在正确设备上
009     model.to(device)
010     # 每个 epoch 验证准确率
011     val_acc_history=[]
012     # 每个 epoch 测试准确率
013     train_acc_history=[]
014     # 每个 epoch 训练损失
015     train_loss_history=[]
016     # 每个 epoch 验证损失
017     val_loss_history=[]
018     # 纪录初始学习率
019     LRs=[optimizer_ft.param_groups[0]['lr']]
020     # 深拷贝当前权重作为初始最佳权重备份
021     best_model_wts=copy.deepcopy(model.state_dict())
022     # 早停计数器（验证集中未提升的连续 epoch 计数）
023     no_improve_count=0
024     # 早停触发信号
025     early_stop=False
026     for epoch in range(num_epochs):
027         if early_stop:
028             print('早停触发，提前结束训练')
029             break
030         print(f'Epoch {epoch}/{num_epochs-1}')
031         print('-' * 10)
032         # 训练和验证
033         for phase in ['train', 'val']:
034             if phase=='train':
035                 model.train()
036             else:
037                 model.eval()
038                 running_loss=0.0
039                 running_corrects=0
040                 # 遍历数据
041                 for inputs, labels in data_loaders[phase]:
042                     inputs=inputs.to(device)
043                     labels=labels.to(device)
```

```

044     # 清零
045     optimizer_ft.zero_grad()
046     # 前向传播: 只训练的时候计算和更新梯度
047     with torch.set_grad_enabled(phase=='train'):
048         outputs=model(inputs)
049         loss=criterion(outputs, labels)
050         _, preds=torch.max(outputs, 1)
051     # 反向传播
052     if phase=='train':
053         loss.backward()
054         optimizer_ft.step()
055     # 计算损失
056     running_loss+=loss.item()*inputs.size(0)
057     running_corrects+=torch.sum(preds==labels.data)
058     # 平均损失和准确率
059     epoch_loss=running_loss/len(data_loaders[phase].dataset)
060     epoch_acc=running_corrects.double()/len(data_loaders[phase].dataset)
061     #计算运行时间
062     time_elapsed=time.time()-since
063     print(f' [phase] Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f} ')
064     # 得到最好结果那次的模型
065     if phase=='val':
066         if epoch_acc>best_acc:
067             best_acc=epoch_acc
068             best_model_wts=copy.deepcopy(model.state_dict())
069             state={
070                 'state_dict':model.state_dict(),
071                 'best_acc':best_acc,
072                 'optimizer_ft':optimizer_ft.state_dict(),
073             }
074             torch.save(state, filename)
075             no_improve_count=0
076             print(f'验证集准确率提升! 保存模型到 {filename} ')
077         else:
078             no_improve_count+=1
079             if no_improve_count>=patience:
080                 early_stop=True
081                 print(f'连续 {patience} 个 epoch 验证集准确率未提升')
082             val_acc_history.append(epoch_acc)
083             val_loss_history.append(epoch_loss)
084         else:
085             train_acc_history.append(epoch_acc)
086             train_loss_history.append(epoch_loss)
087     # 更新学习率 (只在训练阶段后)

```

```

088     scheduler.step()
089     current_lr=optimizer_ft.param_groups[0]['lr']
090     print(f'Optimizer learning rate: {current_lr:.7f}')
091     LRs.append(current_lr)
092     print()
093     time_elapsed=time.time()-since
094     print(f'Training completed in {time_elapsed//60:.0f}m {time_elapsed%60:.0f}s')
095     print(f'Best val acc: {best_acc:.4f}')
096     # 训练完后用最好的一次当作模型最终的结果，用于后续测试
097     model.load_state_dict(best_model_wts)
098     return model, val_acc_history, val_loss_history, LRs, train_acc_history, train_loss_history
099 model_ft, val_acc_history, val_loss_history, LRs, train_acc_history, train_loss_history=train_model(model_fit, data_load
100 ers, criterion, optimizer_ft, scheduler, num_epochs=30, patience=10, filename='resnet18_optimized.pth')

```

2. 多模型对比

(1) 定义多模型初始化函数

为分析不同网络模型在小样本目标分类任务上的表现，本文选取 ResNet18、ResNet50、MobileNetV2 以及 EfficientNet-B0 四种典型卷积神经网络作为对比模型。所有模型均使用原模型提供的预训练权重进行初始化。本文自定模型初始化函数步骤为：

- ◆ 根据模型名称加载对应的预训练模型结构；
- ◆ 调用 set_parameter_requires_grad 函数，根据参数 feature_extract 统一控制模型参数的梯度是否更新；
- ◆ 替换模型顶层分类器，将输出维度修改为 5，确保模型输出层输出统一。

```

01 def get_model(model_name, num_classes, feature_extract=True, use_pretrained=True):
02     """获取指定的预训练模型"""
03     if model_name == 'resnet18':
04         model = models.resnet18(pretrained=use_pretrained)
05         set_parameter_requires_grad(model, feature_extract)
06         num_ftrs = model.fc.in_features
07         model.fc = nn.Linear(num_ftrs, num_classes)
08     elif model_name == 'resnet50':
09         model = models.resnet50(pretrained=use_pretrained)
10         set_parameter_requires_grad(model, feature_extract)
11         num_ftrs = model.fc.in_features
12         model.fc = nn.Linear(num_ftrs, num_classes)
13     elif model_name == 'mobilenet_v2':
14         model = models.mobilenet_v2(pretrained=use_pretrained)
15         set_parameter_requires_grad(model, feature_extract)
16         num_ftrs = model.classifier[1].in_features
17         model.classifier[1] = nn.Linear(num_ftrs, num_classes)
18     elif model_name == 'efficientnet_b0':
19         model = models.efficientnet_b0(pretrained=use_pretrained)
20         set_parameter_requires_grad(model, feature_extract)
21         num_ftrs = model.classifier[1].in_features

```

```

22         model.classifier[1] = nn.Linear(num_fts, num_classes)
23     else:
24         raise ValueError(f'不支持的模型: {model_name}')
25     return model

```

(2) 统一训练模型函数

本文采用两阶段迁移学习策略对模型进行训练。其中，第一阶段采取冻结特征层，仅解冻模型顶层分类层的方法。该阶段目的是避免随机初始化的分类层破坏预训练主干特征，防止前期主干参数大幅度更新引发梯度震荡。结合数据集样本规模，设置最大迭代轮次为 15 轮，早停容忍轮次为 5，验证集准确率连续 5 轮无提升则提前终止训练。优化器用 Adam，初始学习率设置为 0.001，权重衰减系数为 0.0004，搭配余弦退火学习率调度器，学习率下限设置为 0.000006，适配浅层分类层的快速参数更新需求。第二阶段实现全局解冻，微调整个模型。在完成分类头收敛后进入第二阶段全局微调，遍历模型全部参数，将所有主干卷积层、归一化层、分类层梯度求解权限全部开启。基于第一阶段收敛后的分类层参数继续迭代，避免从零开始微调导致过拟合问题。为防止解冻主干后学习率过大覆盖预处理通用特征，本阶段下调学习率至 0.0004，保持权重衰减系数不变，匹配主干网络慢速微调的更新逻辑。同时设置最大迭代轮次为 10 轮，复用相同早停阈值与余弦退火学习率调度策略。浅层分类层与数据集任务强相关，需要较大学习率快速收敛。主干网络承载通过图像特征，需要小学习率细微修正参数，适配目标数据集分布。两阶段训练全部结束后，选取全周期内验证集准确率最高的权重作为模型最终权重，规避后期迭代过拟合风险。单模型训练完成后，自动调用测试集完成模型性能对比评估。调用 get_model_params 函数计算各模型参数量，纪录但模型完整训练耗时，最终统一输出验证准确率、测试准确率、参数量以及训练时长四类对比指标。训练结束后，将各模型最优权重单独保存在文件中。

表 1 多模型对比结果

模型	参数量(M)	验证准确率	测试准确率	训练时间
Efficientnet-B0	4.014	0.978	1.000	25.186
Resnet18	11.180	0.989	0.978	29.646
Resnet50	23.528	0.978	0.978	53.684
Mobilenet-V2	2.230	1.000	0.967	29.550

```

001 def train_single_model(model_name, data_loaders, dataset_sizes, device,
002                         num_classes, feature_extract=True,
003                         num_epochs_stage1=15, num_epochs_stage2=10,
004                         patience=5, verbose=True):
005     """训练单个模型，返回训练结果"""
006     start_time = time.time()
007     # 阶段1: 冻结特征层，只训练分类层
008     model = get_model(model_name, num_classes, feature_extract=True)
009     model = model.to(device)
010     # 获取待训练参数
011     params_to_update = []
012     for name, param in model.named_parameters():
013         if param.requires_grad:
014             params_to_update.append(param)
015     optimizer = optim.Adam(params_to_update, lr=1e-3, weight_decay=1e-4)

```

```

016 #余弦退火
017 scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=num_epochs_stage1, eta_min=1e-6)
018 #损失函数
019 criterion = nn.CrossEntropyLoss()
020 # 训练阶段 1
021 best_acc_stage1 = 0.0
022 best_model_wts_stage1 = copy.deepcopy(model.state_dict())
023 no_improve_count = 0
024 for epoch in range(num_epochs_stage1):
025     for phase in ['train', 'val']:
026         if phase == 'train':
027             model.train()
028         else:
029             model.eval()
030             running_loss = 0.0
031             running_corrects = 0
032             for inputs, labels in data_loaders[phase]:
033                 inputs, labels = inputs.to(device), labels.to(device)
034                 optimizer.zero_grad()
035                 with torch.set_grad_enabled(phase == 'train'):
036                     outputs = model(inputs)
037                     loss = criterion(outputs, labels)
038                     _, preds = torch.max(outputs, 1)
039                     if phase == 'train':
040                         loss.backward()
041                         optimizer.step()
042                     running_loss += loss.item() * inputs.size(0)
043                     running_corrects += torch.sum(preds == labels.data)
044             epoch_loss = running_loss / dataset_sizes[phase]
045             epoch_acc = running_corrects.double() / dataset_sizes[phase]
046             if phase == 'val' and epoch_acc > best_acc_stage1:
047                 best_acc_stage1 = epoch_acc
048                 best_model_wts_stage1 = copy.deepcopy(model.state_dict())
049                 no_improve_count = 0
050             elif phase == 'val':
051                 no_improve_count += 1
052             scheduler.step()
053             if verbose and (epoch + 1) % 3 == 0:
054                 print(f' Epoch {epoch+1}/{num_epochs_stage1}, Val Acc: {best_acc_stage1:.4f}')
055             if no_improve_count >= patience:
056                 if verbose:
057                     print(f' 阶段1 早停, epoch {epoch+1}')
058                 break
059 # 阶段2: 解冻全部层, 微调

```

```

060 # 解冻所有层
061 for param in model.parameters():
062     param.requires_grad = True
063 model.load_state_dict(best_model_wts_stage1)
064 optimizer_ft = optim.Adam(model.parameters(), lr=1e-4, weight_decay=1e-4)
065 scheduler_ft = optim.lr_scheduler.CosineAnnealingLR(optimizer_ft, T_max=num_epochs_stage2, eta_min=1e-6)
066 # 训练阶段 2
067 best_acc = best_acc_stage1
068 best_model_wts = copy.deepcopy(best_model_wts_stage1)
069 no_improve_count = 0
070 for epoch in range(num_epochs_stage2):
071     for phase in ['train', 'val']:
072         if phase == 'train':
073             model.train()
074         else:
075             model.eval()
076             running_loss = 0.0
077             running_corrects = 0
078             for inputs, labels in data_loaders[phase]:
079                 inputs, labels = inputs.to(device), labels.to(device)
080                 optimizer_ft.zero_grad()
081                 with torch.set_grad_enabled(phase == 'train'):
082                     outputs = model(inputs)
083                     loss = criterion(outputs, labels)
084                     _, preds = torch.max(outputs, 1)
085                     if phase == 'train':
086                         loss.backward()
087                     optimizer_ft.step()
088                 running_loss += loss.item() * inputs.size(0)
089                 running_corrects += torch.sum(preds == labels.data)
090             epoch_loss = running_loss / dataset_sizes[phase]
091             epoch_acc = running_corrects.double() / dataset_sizes[phase]
092             if phase == 'val' and epoch_acc > best_acc:
093                 best_acc = epoch_acc
094                 best_model_wts = copy.deepcopy(model.state_dict())
095                 no_improve_count = 0
096             elif phase == 'val':
097                 no_improve_count += 1
098             scheduler_ft.step()
099             if verbose and (epoch + 1) % 3 == 0:
100                 print(f' Epoch {epoch+1}/{num_epochs_stage2}, Val Acc: {best_acc:.4f}')
101             if no_improve_count >= patience:
102                 if verbose:
103                     print(f' 阶段 2 早停, epoch {epoch+1}')

```

```

104     break
105     # 加载最佳模型
106     model.load_state_dict(best_model_wts)
107     # 在测试集上评估
108     model.eval()
109     test_corrects = 0
110     with torch.no_grad():
111         for inputs, labels in data_loaders['test']:
112             inputs, labels = inputs.to(device), labels.to(device)
113             outputs = model(inputs)
114             _, preds = torch.max(outputs, 1)
115             test_corrects += torch.sum(preds == labels.data)
116     test_acc = test_corrects.double() / dataset_sizes['test']
117     total_time = time.time() - start_time
118     return {
119         'model_name': model_name,
120         'best_val_acc': best_acc.item() if torch.is_tensor(best_acc) else best_acc,
121         'test_acc': test_acc.item() if torch.is_tensor(test_acc) else test_acc,
122         'params_M': get_model_params(model),
123         'training_time_min': total_time / 60,
124         'model': model,
125         'best_model_wts': best_model_wts
126     }

```

(3) YOLOv8 模型

针对实际药材图片识别多为多分类任务问题，本文首先尝试使用 YOLO 来对单样本图片进行识别，便于后续处理多样本问题。针对图片识别问题，YOLO 具有目标定位和类别判别能力。故本实验将整张图片作为单一目标进行标注，通过全局边界框方式使模型能够判别图像所属类别。为适配电脑性能，本文使用轻量级的 YOLOV8 来实现这一任务。为适配 YOLO 系列算法输入规范，本文将原有分类数据集转换为 YOLO 标准格式。在图片标注过程中，本文将整张图片作为单一目标，标注坐标设为(0.5 0.5 1.0 1.0)，以此把分类任务转化为目标检测任务。在模型训练过程中，本文加载预训练权重以展开迁移学习。训练过程实时监测边界框损失、分类损失、dfl 损失以及精度指标。从实验结果可以观察发现，该模型收敛速度快，训练前期损失快速下降，分类损失由 1.895 降至 0.3306，边界框损失由 0.3391 降至 0.0599，对应 mAP@0.5 在第 2 轮达到 0.925。训练中后期损失趋于平稳，指标小范围波动并稳步提升，在第 9 轮后 mAP@0.5 稳定在 0.97 以上，模型整体训练状态稳定，没有出现明显欠拟合或严重过拟合现象。

类别	样本数	精确率	召回率	mAP@0.5	mAP@0.5:0.95
百合	18	0.895	1.000	0.995	0.995
党参	19	0.934	1.000	0.990	0.990
枸杞	18	1.000	0.972	0.995	0.995
槐花	17	1.000	0.887	0.955	0.955
金银花	18	0.986	1.000	0.995	0.995
总体	90	0.963	0.972	0.986	0.986


```

01 # 训练模型
02 results = yolo_model.train(
03     data=os.path.join(yolo_output_dir, 'data.yaml'),
04     epochs=20,
05     batch=8,
06     imgsz=416,
07     device='cpu',
08     optimizer='Adam',
09     lr0=1e-3,
10     weight_decay=1e-4,
11     verbose=True,
12     name='yolov8n_herb'
13 )
14 # 保存模型
15 yolo_model.save('yolov8n_herb.pt')
16 print('YOLOv8 模型训练完成')
17 val_results = yolo_model.val(data=os.path.join(yolo_output_dir, 'data.yaml'))

```

(4) 自定义图片测试

本文使用上面图片识别效果最好的 EfficientNet-B0 对样本数据集以外的图片进行识别。首先，系统加载训练完成的 EfficientNet-B0 权重文件，对经过尺寸缩放、张量转换以及标准化的图片进行分类。

识别结果：枸杞
置信度：98.82%



图 4 测试图片结果

```

01 def load_trained_model(model_path, num_classes, device):
02     model = models.efficientnet_b0(pretrained=False)
03     print(f"原始 classifier 结构: {model.classifier}")
04     num_fters = model.classifier[1].in_features
05     model.classifier[1] = nn.Linear(num_fters, num_classes)
06     model.load_state_dict(torch.load(model_path, map_location=device))

```

```

07     model = model.to(device)
08     model.eval()
09     print(f"EfficientNet-B0 模型加载成功! 分类类别数: {num_classes}")
10     return model
11 test_transform = transforms.Compose([
12     transforms.Resize([224, 224]),
13     transforms.ToTensor(),
14     transforms.Normalize(
15         mean=[0.485, 0.456, 0.406],
16         std=[0.229, 0.224, 0.225]
17     )
18 ])
19 def predict_herb(image_path, model, class_names, device):
20     image = Image.open(image_path).convert('RGB')
21     input_tensor = test_transform(image).unsqueeze(0).to(device)
22     with torch.no_grad():
23         outputs = model(input_tensor)
24         probs = torch.softmax(outputs, dim=1)
25         pred_idx = torch.argmax(probs, dim=1).item()
26         pred_class = class_names[pred_idx]
27         confidence = probs[0][pred_idx].item()
28     return pred_class, confidence, image

```

五、实验结果分析

1. 单模型训练结果

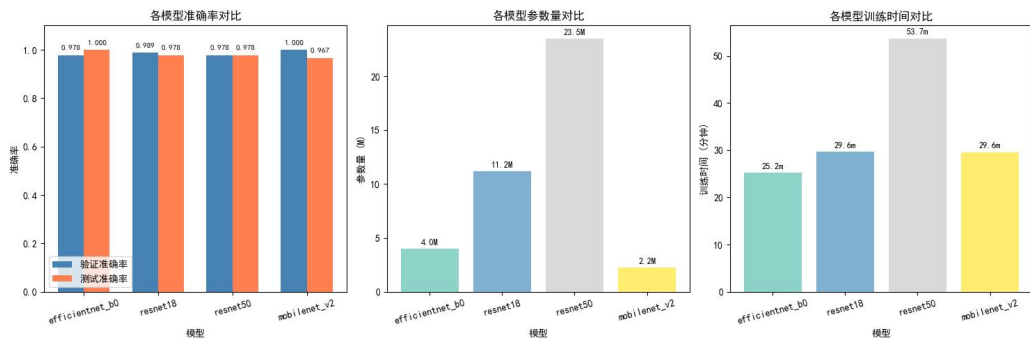


图 5 单模型训练结果图

从图片中我们可以观察发现，本文使用 ResNet18 模型收敛速度快，在前期迭代中训练集和验证集模型准确率均快速提升，损失值迅速下降。在第 12 个 epoch 左右，训练集准确率稳定在 0.975 以上，验证集准确率也稳定在 0.95 附近，说明此时模型已经基本收敛，继续训练对模型整体性能提升有限，故模型在第 18epoch 早停。损失曲线在后期也维持在较低水平，波动浮动明显减小，表明模型的参数更新趋于平稳。对比训练集和验证集后期的准确率发现，两者差距约为 1%~4%之间，这一差距说明模型存在轻微过拟合现象。但是早停机制的引入有效遏制了过拟合的进一步发展，保障了模型的泛化能力。

2. 多模型对比结果

观察图片结果发现，模型整体表现优异，验证准确率均在 97% 以上，测试准确率也在 96.7%~100% 之间，说明所有模型在本次分类任务中都具备较强的图片识别能力。其中，EfficientNet-B0 和 ResNet18 的验证集与测试集准确率差距较小，说明模型泛化能力较好，未出现明显过拟合现象。MobileNet 的测试准确率略低于验证准确率，这说明模型可能存在轻微过拟合，但其整体性能仍处于较高水平。观察模型参数对比发现，MobileNetV2 参数量仅为 2.2M，EfficientNet-B0 为 4.0M，而 ResNet18 为 11.2M，ResNet50 高达 23.5M。EfficientNet-B0 在较少参数量的条件下，实现了和其它模型相当的准确率，这说明该模型具有较高的模型效率。而 ResNet50 参数量最大，但是性能并未显著优于其它模型，这说明在该任务中，增加模型深度带来的性能提升已趋于饱和。训练时间对比结果发现，模型训练时间长短与模型参数量基本呈正相关关系。模型参数量越大，模型的训练时间也就越长。



六、智能体调用

为了使模型对图片识别项目能够轻落地，本文搭建了“本草小药师”智能体以实现根据识别结果智能回答该药材的简介、药材对比以及用户用药个性化推荐等相关内容。该阶段主要通过智能体搭建和智能体 api 调用两个部分。

1. 智能体搭建

(1) 知识库搭建

查询专业网站与五位药材的相关介绍，根据实际搭建知识库。以下是知识库关于党参药材的相关介绍：

- 【药材名称】** 枸杞
- 【别名】** 枸杞子、红耳坠、血杞子、明目子
- 【来源】** 茄科植物宁夏枸杞（*Lycium barbarum*）的干燥成熟果实
- 【产地】** 主产于宁夏、甘肃、青海、新疆、内蒙古等地。以宁夏中宁所产品质最佳，为地道药材。
- 【采收加工】** 夏秋二季果实呈红色时采收，热风烘干或晾至皮皱后晒干。
- 【药性】** 味甘，性平。归肝经、肾经。
- 【功效】** 滋补肝肾，益精明目，润肺止咳。
- 【主治】** 虚劳精亏，腰膝酸痛，眩晕耳鸣，内热消渴，血虚萎黄，目昏不明，视力减退。
- 【用法用量】** 煎服，6~12g。亦可泡茶、泡酒、生食、炖汤。
- 【副作用与禁忌】**
- （1）脾虚便溏者慎服——枸杞有润肠作用，便溏者服用可能加重腹泻。
 - （2）外邪实热者忌用——感冒发热期间不宜食用。
 - （3）不宜过量食用——每日超过 30g 可能出现上火症状，如口干、流鼻血等。
 - （4）高血压患者在血压波动期慎用。
- 【经典配伍】**
- 枸杞配菊花——滋补肝肾、清肝明目，经典对药。

- 【趣味知识】《本草纲目》记载枸杞“久服坚筋骨，轻身不老，耐寒暑”。宁夏枸杞被列为道地药材之首，有“天下枸杞出宁夏”之说。现代研究发现枸杞多糖对视网膜细胞有保护作用，印证了古人“明目”的经验。

提示词首先定义智能体角色为“本草小药师”，明确专门介绍党参、枸杞、百合、金银花、槐花五位中药材，知识来源于权威中医药文献，要求用通俗语言介绍并引导用户前往专业查询平台。接着，定义智能体的四项核心能力：药材简介查询、药材对比、体质推荐等。然后，插入联网问答插件，便于智能体无法在智能体查询相关信息时能够联网搜索相关内容。最后，指定智能体行为约束，要求智能体只能回答与五类药材相关的内容以及不能提供具体的医疗处方或诊断建议。

智能体搭建完成并发布后，通过智能体的 **api** 对其进行调用。本文将上面识别结果“枸杞”返回智能体，得到如下结果：

中药材的使用请在专业中医师指导下进行，切勿自行用药或擅自调整用量。

```
01 def get_herb_info(herb_name):
02     try:
03         client = Coze(
04             auth=TokenAuth(token=COZE_API_TOKEN),
05             base_url="https://api.coze.cn"
06         )
07         chat = client.chat.create(
08             bot_id=COZE_BOT_ID,
```

	<pre> 09 user_id="herb_user_001", 10 additional_messages=[11 Message.build_user_question_text(12 f'请介绍中药材{herb_name}的功效、用法和注意事项' 13) 14]) 15 full_answer = "" 16 for event in client.chat.stream(17 bot_id=COZE_BOT_ID, 18 user_id="herb_user_001", 19 conversation_id=chat.conversation_id, 20 chat_id=chat.id 21): 22 if event.event == ChatEventType.CONVERSATION_MESSAGE_DELTA: 23 content = event.message.content or "" 24 full_answer += content 25 elif event.event == ChatEventType.CONVERSATION_CHAT_COMPLETED: 26 break 27 return full_answer.strip() if full_answer else "未获取到智能体回复内容" 28 except Exception as e: 29 return f"请求异常: {str(e)}" </pre>
心得体会	通过本次课程设计，我系统地学习了从数据处理、模型训练、智能体搭建到系统集成的完整流程，对深度学习在图像分类与目标检测任务中的应用有了更加深入的理解。在模型训练阶段，通过对比 ResNet18、ResNet50、MobileNetV2 和 EfficientNet-B0 四种模型，我发现 EfficientNet-B0 在参数量最小的条件下取得最好的测试准确率，说明其在小样本场景下的高效性。ResNet50 虽参数量最大，但其性能并未显著优于其它模型，说明在数据量有限的情况下，单纯增加模型深度并不能带来明显的性能提升。迁移学习以及早停机制等策略的引入，有效地缓解了小样本训练的过拟合问题，两阶段训练的策略使得模型能够稳定收敛。在智能体搭建阶段，我学习了 coze 平台包括知识库构建、提示词撰写、智能体发布等环节的使用方法。本次实验让我认识到，将深度学习技术应用于中医药这一传统领域，不仅需要扎实的模型训练能力，还需要对应用场景的深入理解。本次实验我收获颇丰。
教师评语	